ConnectHub

PROJECT OVERVIEW

The **ConnectHub** is a backend web application designed to solve the common problem of scattered contact information across multiple platforms. It provides users with a centralized, organized system to store, search, and categorize their professional and personal contacts.

Key Objectives:

- Provide secure user authentication and data isolation
- Enable efficient contact management through CRUD operations
- Implement flexible categorization using tags
- Offer fast search and filtering capabilities
- Deliver a responsive, cross-device user experience

Target Users:

- Freelancers and consultants managing client relationships
- Small business owners tracking vendor and customer contacts
- Students and young professionals building their network
- Event organizers managing attendee and volunteer information
- Anyone seeking a simple, privacy-focused contact management solution

PROJECT INSTRUCTION

1️⃣ Install Node.js (If Not Installed)

Download and install Node.js from: https://nodejs.org/

After installation, verify:

```
node -v
npm -v
```

2️⃣ Create a Backend Project Folder

- Choose or create a directory where you want your backend.
- Open your terminal or command prompt.
- Navigate to the selected directory:

```
cd your-project-folder
```

- Create a new backend folder:

```
mkdir server
cd server
```

3️⃣ Initialize Node Project

Run: `npm init -y`

This will create a `package.json` file.

4️⃣ Install Required Dependencies

Install Express:

```
npm install express
npm install mongoose
```

For development (auto-restart server):

```
npm install nodemon --save-dev
```

5️⃣ Create Basic Server File

Inside the `server` folder:

- Create a file called: **`server.js`**

`Add this code :`

```
import express from "express";

const app = express();
const PORT = 5000;

// Middleware
app.use(express.json());

// Test Route
app.get("/", (req, res) => {
  res.send("Backend is running 🚀");
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

6️⃣ Enable ES Modules (Important)

Open package.json and add: **"type": "module"**

Example:

```json
{
  "name": "server",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon server.js"
  }
}
```

7️⃣ Start the Backend Server

Run: `npm run dev`
OR if not using nodemon: `node server.js`

8️⃣ Open in Browser

Open: `http://localhost:5000`
You should see: `Backend is running`

PRE-REQUISITES

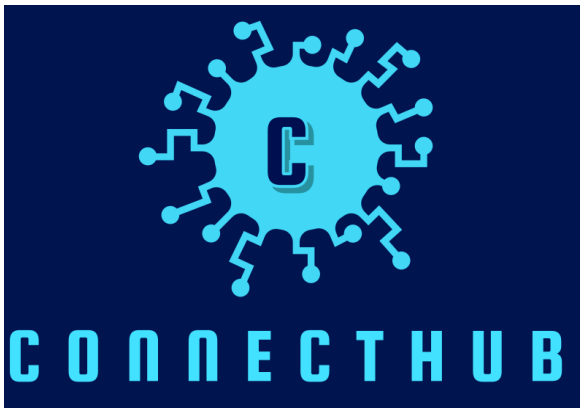
Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code, WebStorm, or similar

Knowledge Requirements

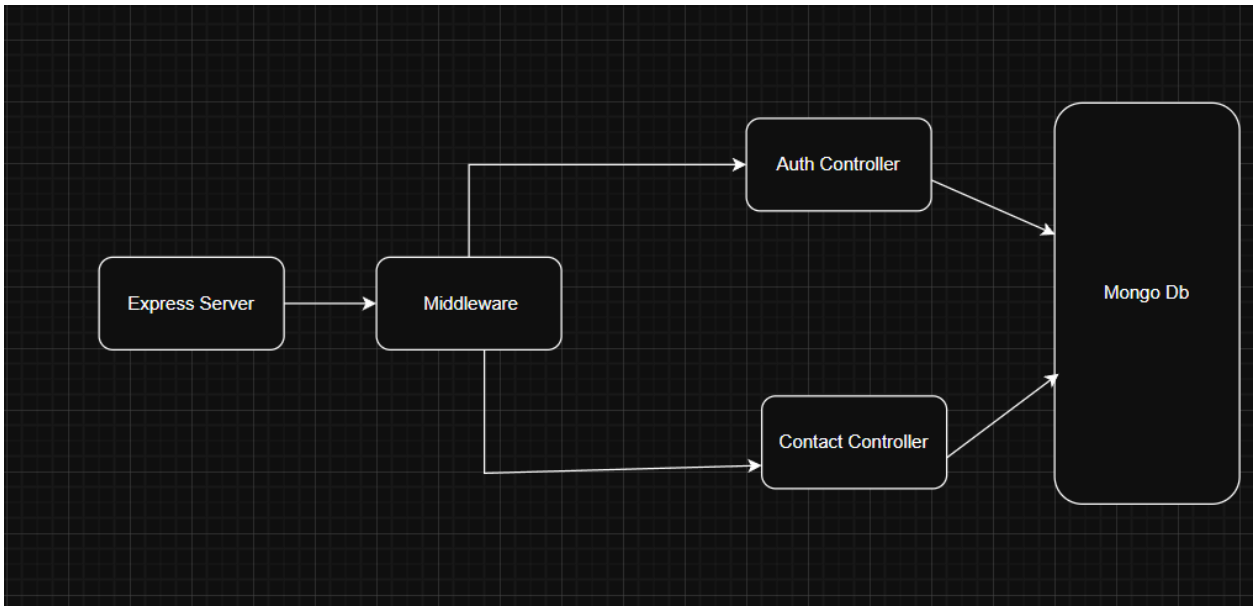
- JavaScript ES6+ syntax and concepts
- Node.js and Express.js for RESTful API development
- MongoDB and Mongoose ODM
- JWT authentication and bcrypt password hashing
- Git version control workflow

PROJECT THUMBNAIL



PROJECT ARCHITECTURE

High-level architecture diagram illustrating the three-tier backend structure.



Node.js + Express Backend

• RESTful API architecture
• JWT authentication middleware
• MVC pattern (Models, Controllers, Routes)
• Input validation and error handling

MongoDB Database

• Document-based NoSQL storage
• Mongoose ODM for schema modeling
• Two collections: Users and Contacts
• Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

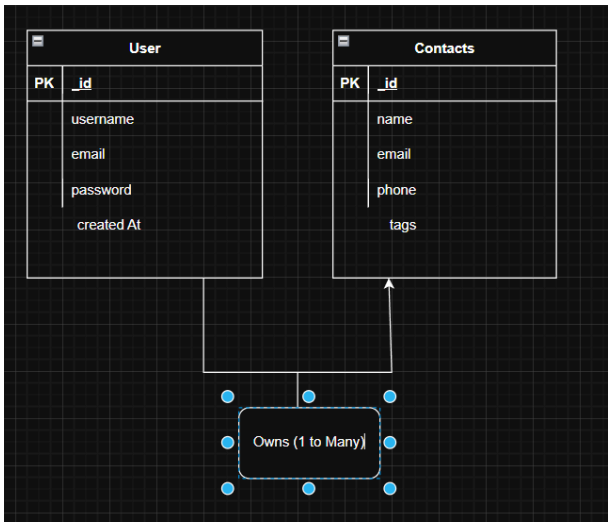
Backend Architecture (Node.js + Express)

- **Controllers:** Business logic for authentication and CRUD operations
- **Models:** Mongoose schemas defining data structure
- **Routes:** API endpoint definitions with HTTP methods
- **Middleware:** JWT verification, error handling, validation
- **Configuration:** Database connection and environment variables

Database Architecture (MongoDB)

- **Users Collection:** _id, name, email, password (hashed), timestamps
- **Contacts Collection:** _id, userId (FK), name, email, phone, notes, tags[], timestamps
- **Relationships:** Many-to-One (Contacts → User)
- **Indexes:** email (unique), userId, name, tags (for search)

ER Diagram



Features

Core Features

#	Feature	Description

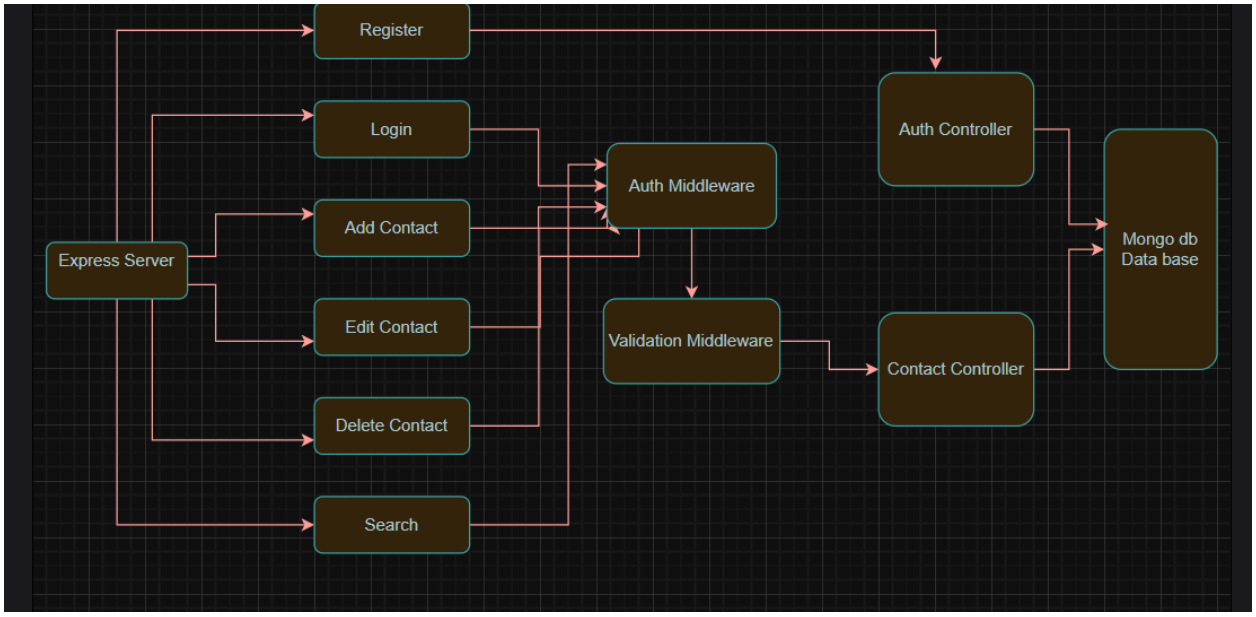
1	User Authentication	JWT-based registration, login, and session management with bcrypt password hashing
2	Contact CRUD	Create, read, update, and delete contacts with full validation and error handling
3	Tag Categorization	Organize contacts with multiple tags (Work, Family, Friends, etc.) and filter by category
4	Real-time Search	Debounced search across name, email, and tags with instant results

Roles and Responsibility

- User -

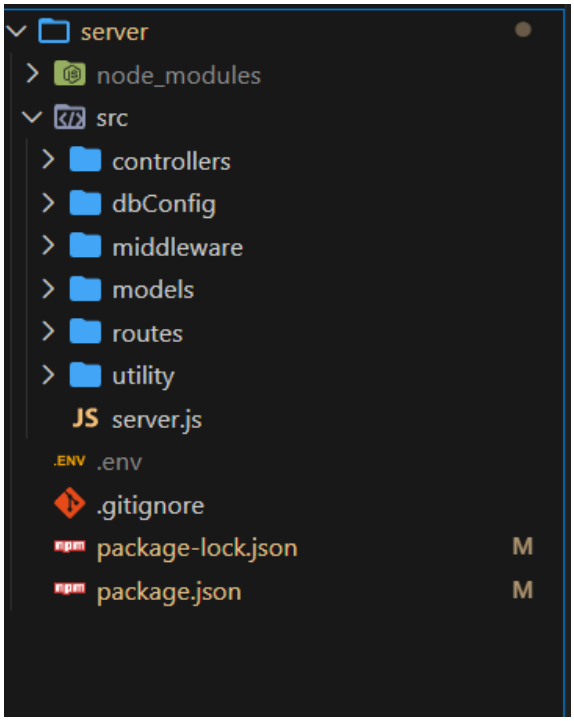
    1. Registration - Register on the application using the name, email , password
    2. Login - Login on the application using the email and registered password
    3. Search - Search for a contact using name , email or tags
    4. Add Contact - Add new contact - name , email , phone , notes , tags
    5. Edit Contact - Edit the existing contact
    6. Delete Contact - Delete any contact

User Flow



Project Setup and Configuration

Folder Structure



Project Setup

1. Install required tools and software:

- Node.js.
- MongoDB.

2. Create project folders and files:
   - Server folders.

3. Install Packages:

   Backend npm Packages
   - Express.
   - Mongoose.
   - Cors.

Backend Development

- Setup express server
  - Create index.js file in the server (backend folder).
  - Create a .env file and define port number to access it globally.
  - Configure the server by adding cors, body-parser.

```
// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
```

- User Authentication:
  - Create routes and middleware for user registration, login, and
  - Set up authentication middleware to protect routes that require user authentication.

```
import express from 'express';
const router = express.Router();
import { registerUser, loginUser, getMe } from '../controllers/authController.js';
import  authentication  from '../middleware/authMiddleware.js';
import { registerValidation, loginValidation } from '../utility/validators.js';
import validate from '../middleware/validation.js';

router.post('/register', registerValidation, validate, registerUser);
router.post('/login', loginValidation, validate, loginUser);
router.get('/me', authentication, getMe);

export default router;
```

- Define API Routes:
  - Create separate route files for different API functionalities such as user authentication and crud operation for contacts
  - Implement route handlers using Express.js to handle requests and interact with the database.

```
router.get('/search', authentication, searchContacts);
router.route('/')
  .get(authentication, getContacts)
  .post(authentication, contactValidation, validate, createContact);

router.route('/:id')
  .get(authentication, getContactById)
  .put(authentication, updateContactValidation, validate, updateContact)
  .delete(authentication, deleteContact);
```

- Implement Data Models:
  - Define Mongoose schemas for the different data entities like user and contact
  - Create corresponding Mongoose models to interact with the MongoDB database.
  - Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

```
import mongoose from "mongoose";
import bcrypt from 'bcryptjs'
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please add a name'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Please add an email'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email',
      ],
    },
    password: {
      type: String,
      required: [true, 'Please add a password'],
      minlength: 6,
      select: false,
    },
  },
  {
    timestamps: true,
  }
);

// Hash password before saving
```

```
const contactSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      required: true,
      ref: 'User',
    },
    name: {
      type: String,
      required: [true, 'Please add a contact name'],
      trim: true,
    },
    email: {
      type: String,
      trim: true,
      lowercase: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email',
      ],
    },
    phone: {
      type: String,
      trim: true,
    },
    tags: {
      type: [String],
      default: [],
    },
  },
  {
    timestamps: true,
  }
);
```

🎬 Error Handling:

- Implement error handling middleware to catch and handle any errors that occur during the API requests.
- Return appropriate error responses with relevant error messages and HTTP status codes.

```
const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack,
  });
};

const notFound = (req, res, next) => {
  const error = new Error(`Not Found - ${req.originalUrl}`);
  res.status(404);
  next(error);
};

export { errorHandler, notFound };
```

Database Configuration and Setup

1. Configure MongoDB:
   - Install Mongoose.
   - Create database connection.
   - Create Schemas & Models.

2. Connect database to backend:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

```
const PORT = process.env.PORT || 6001;
mongoose.connect(process.env.MONGO_URL, {
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(()=>{


    server.listen(PORT, ()=>{
        console.log(`Running @ ${PORT}`);
    });


}).catch((err)=>{
    console.log("Error: ", err);
})
```

3. Configure Schema:

Firstly, configure the Schemas for MongoDB database, to store the data in such pattern.
Use the data from the ER diagrams to create the schemas.
The schemas are looks like for the Application.

```
User.js
server > models > User.js > ...
import mongoose from 'mongoose';

const UserSchema = new mongoose.Schema({
    username:{
        type: String,
        require: true
    },
    email:{
        type: String,
        require: true,
        unique: true
    },
    password:{
        type: String,
        require: true
    },
});

const User = mongoose.model("users", UserSchema);
export default User;
```

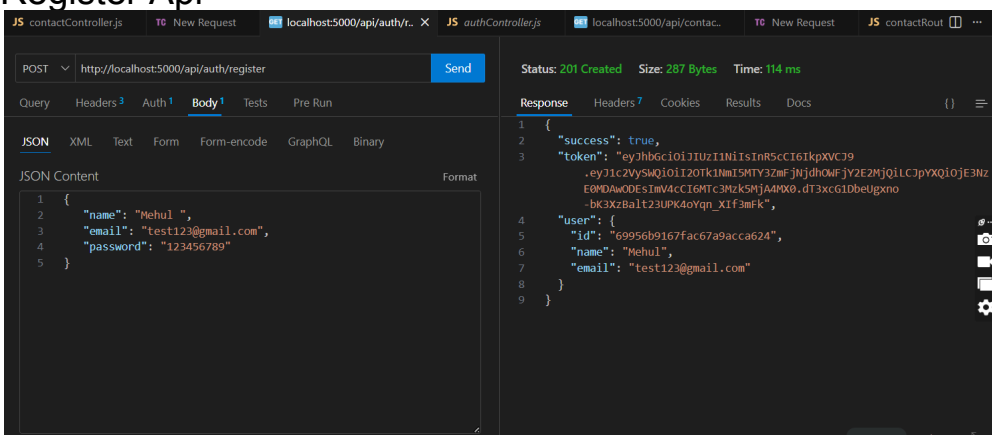
Project Execution

Execution Steps

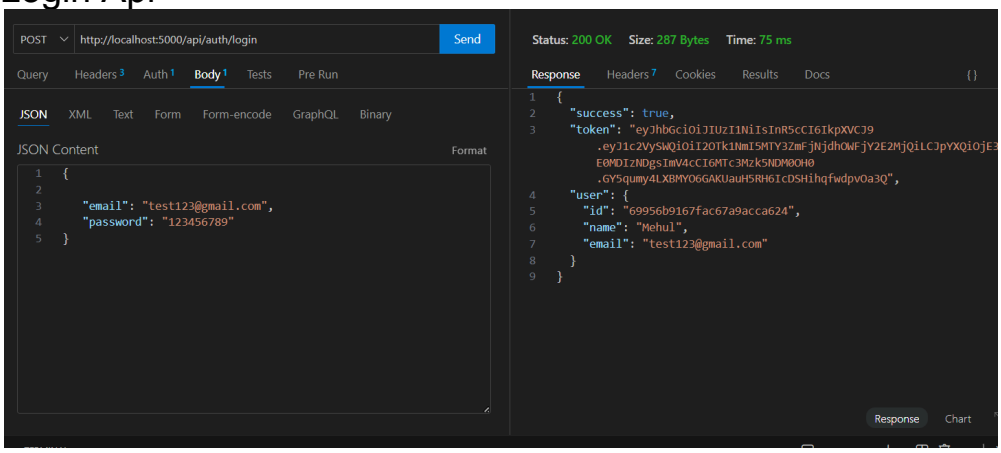
1. Open the project folder in the terminal
2. Make sure you have a start script in the package.js file
3. Use the command **npm start** to start the server
4. You will the **Running on PORT** message on the successfully starting the server
5. You will also see the database connected message if the database is connected successfully
6. Use the Thunder client or Postman for using the created api's

Output Screenshots

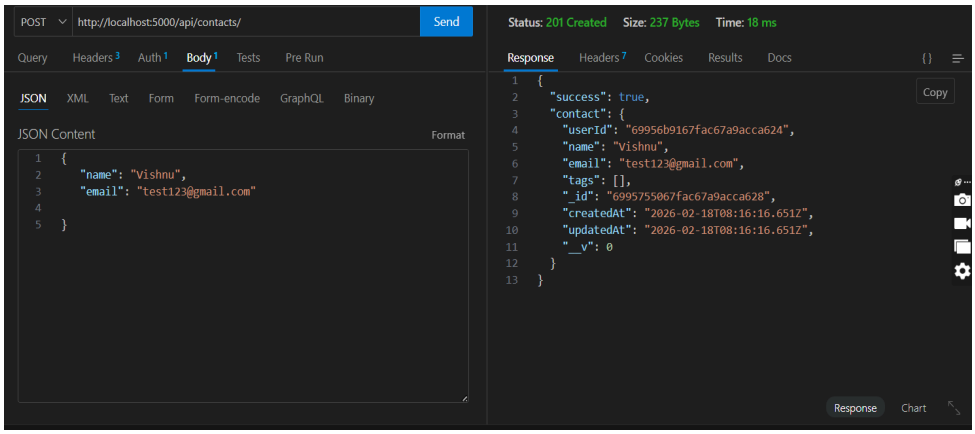
1. Register Api



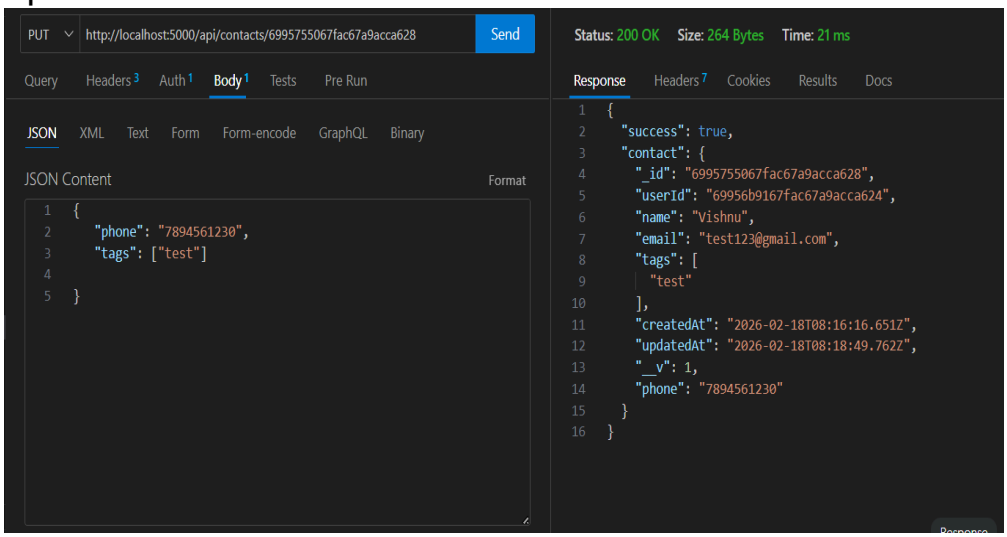
2. Login Api



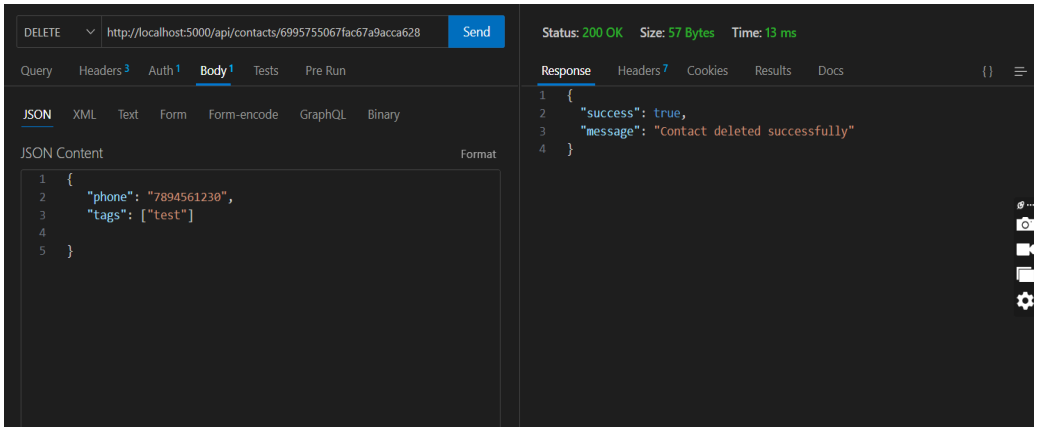
3. Create New Contact



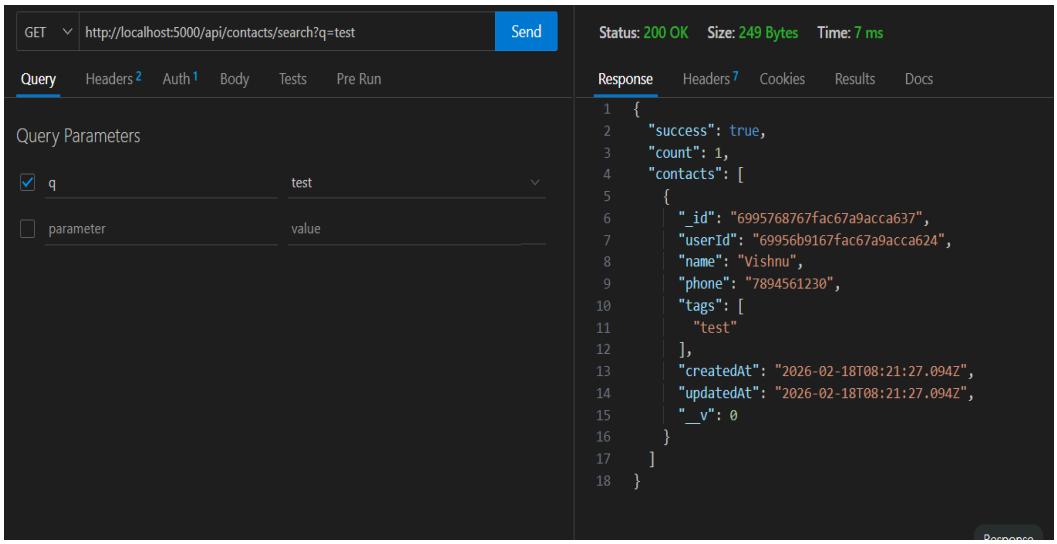
4. Update Contact



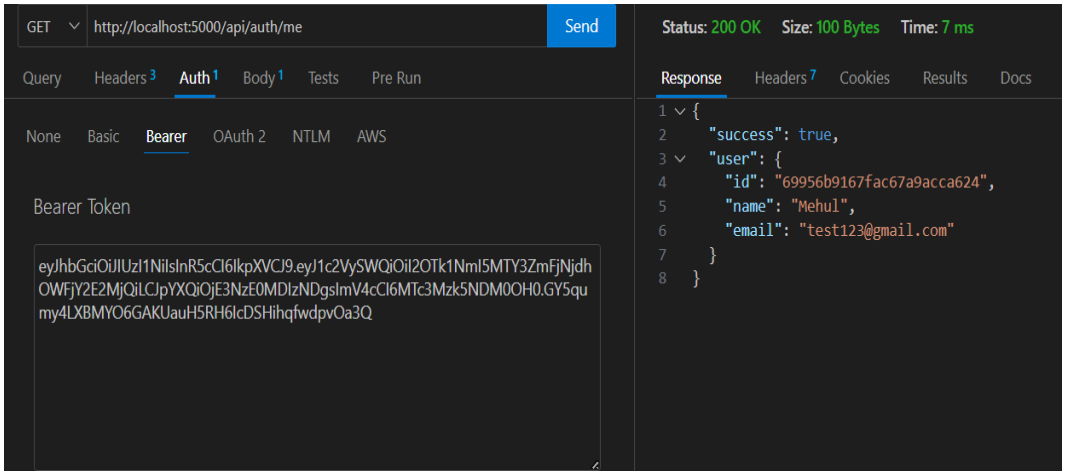
5. Delete Contact



6. Search Contact



7. Get The Current User



Project Demo
[Contact Manager.webm]

Code Link

[Contact-Manager]